LeetCode – Nov 2026 | Day 5

Author: Eric Zhou
Email: eric_ds_zhou@outlook.com
Date: Nov 12, 2026

Introduction

Today's practice focuses on two **in-place array manipulation** problems: LeetCode 75 and LeetCode 48.

LeetCode 75 uses the **Three Pointers** technique to sort an array containing only three distinct values in a single pass. LeetCode 48 uses **Matrix Transposition and Row Reversal** to rotate a square matrix by 90 degrees clockwise.

Both problems modify the original data structure directly without allocating another array or matrix.

Leetcode 75 - Sort Colors

Question

Given an integer array `nums` containing `n` objects colored red, white, or blue, sort them **in-place** so that objects of the same color are adjacent, in the order red, white, and blue.

The colors are represented by the integers:

- `0`: Red
- `1`: White
- `2`: Blue

You must solve this problem without using the library's sorting function.

Example:

Input: `nums = [2,0,2,1,1,0]`
Output: `[0,0,1,1,2,2]`

Intuition

Since the array contains only three distinct values, we do not need a general sorting algorithm.

Instead, we can use the **Three Pointers** technique, also known as the **Dutch National Flag Algorithm**, to divide the array into three regions.

Our goal is to move all `0`s to the left and all `2`s to the right. The `1`s will naturally remain in the middle.

We can accomplish this by swapping elements in-place while traversing the array.

Approach

We use three pointers:

- `low`: Indicates the next position where a `0` should be placed.
- `mid`: Indicates the current element being examined.
- `high`: Indicates the next position where a `2` should be placed.

Initially:

```
int low = 0;
int mid = 0;
int high = nums.length - 1;
```

During the algorithm, we maintain four regions:

- `[0, low - 1]`: Contains only `0`s.
- `[low, mid - 1]`: Contains only `1`s.
- `[mid, high]`: Contains elements that have not been processed.
- `[high + 1, n - 1]`: Contains only `2`s.

While `mid <= high`, we consider three cases.

Case 1: `nums[mid] == 0`

Swap `nums[mid]` with `nums[low]`, then increment both `low` and `mid`.

The element moved to `mid` is already processed, so we can safely move forward.

Case 2: `nums[mid] == 1`

The current element belongs in the middle region. No swap is necessary, so we simply increment `mid`.

Case 3: `nums[mid] == 2`

Swap `nums[mid]` with `nums[high]`, then decrement `high`.

However, we **do not increment** `mid`, because the element swapped from the right side has not been examined yet.

The algorithm finishes when `mid > high`, meaning all elements have been processed.

Time Complexity

$$\boxed{\Theta(n)}$$

Each iteration processes at least one element by either incrementing `mid` or decrementing `high`.

Since each pointer moves in only one direction and every operation takes constant time, the total time complexity is $\Theta(n)$.

Space Complexity

$$\boxed{\Theta(1)}$$

We modify the input array in-place using only three pointers and one temporary variable for swapping.

No additional array is allocated, so the extra space complexity is $\Theta(1)$.

Code

```
class Solution {
    public void sortColors(int[] nums) {
        int temp;
        int low = 0;
        int mid = 0;
        int high = nums.length - 1;

        while (mid <= high) {
            if (nums[mid] == 0) {
                temp = nums[mid];
                nums[mid] = nums[low];
                nums[low] = temp;
                low++;
                mid++;
            } else if (nums[mid] == 2) {
                temp = nums[mid];
                nums[mid] = nums[high];
                nums[high] = temp;
                high--;
            } else {
                mid++;
            }
        }
    }
}
```

Leetcode 48 - Rotate Image

Question

Given an `n x n` 2D matrix representing an image, rotate the image by **90 degrees clockwise**.

You must rotate the matrix **in-place**, meaning that you must modify the original matrix directly without allocating another 2D matrix.

Example:

Input:

```
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
```

Output:

```
[7, 4, 1]
[8, 5, 2]
[9, 6, 3]
```

Intuition

Instead of directly moving every element to its final position, we can divide the rotation into two simpler operations:

1. **Transpose the Matrix**
2. **Reverse Each Row**

Transposing the matrix swaps the rows and columns. Reversing each row then produces the desired 90-degree clockwise rotation.

Both operations can be performed in-place using only a temporary variable.

Approach

Step 1: Transpose the Matrix

For an `n x n` matrix, transposition exchanges each element `matrix[i][j]` with `matrix[j][i]`.

We only need to process elements above the main diagonal, because swapping every pair twice would restore the original matrix.

Therefore, the inner loop starts from `j = i + 1`.

```
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        int temp = matrix[i][j];
        matrix[i][j] = matrix[j][i];
        matrix[j][i] = temp;
    }
}
```

For example:

Original matrix:

```
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
```

After transposition:

```
[1, 4, 7]
[2, 5, 8]
[3, 6, 9]
```

Step 2: Reverse Each Row

After transposing the matrix, reverse every row by swapping the elements symmetrically.

For each row `i`, swap `matrix[i][j]` with `matrix[i][n - j - 1]`.

We only need to traverse the first half of each row, so the inner loop condition is `j < n / 2`.

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n / 2; j++) {
        int temp = matrix[i][j];
        matrix[i][j] = matrix[i][n - j - 1];
        matrix[i][n - j - 1] = temp;
    }
}
```

After reversing each row:

```
[7, 4, 1]
[8, 5, 2]
[9, 6, 3]
```

The matrix has now been rotated by 90 degrees clockwise.

Time Complexity

$$\boxed{\Theta(n^2)}$$

Transposing the matrix requires swapping approximately half of the off-diagonal elements, which takes $\Theta(n^2)$ time.

Reversing each row requires $\lfloor n/2 \rfloor$ swaps for each of the `n` rows, which also takes $\Theta(n^2)$ time.

Therefore, the total time complexity is:

$$\Theta(n^2) + \Theta(n^2) = \boxed{\Theta(n^2)}$$

Space Complexity

$$\boxed{\Theta(1)}$$

Both matrix transposition and row reversal are performed in-place.

We only use loop variables and a temporary variable for swapping, so the extra space complexity is $\Theta(1)$.

Code

```java
class Solution {
    public void rotate(int[][] matrix) {
        int n = matrix.length;

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n / 2; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[i][n - j - 1];
                matrix[i][n - j - 1] = temp;
            }
        }
    }
}
```